NGSolve Agent - журнал разработки

Этот файл - рабочий журнал проекта: что сделано, почему, как проверено и что делать дальше. Записи идут от новых к старым; после каждого существенного изменения кода или запуска добавляется новая запись.

1 Запись 2026-11-10 - реальные LLM-адаптеры для D1/D4

1.1 Что изменено

I. Добавлен src/ngsolve_agent/llm_adapter.py: один JSON-only адаптер поддерживает Ollama local chat и OpenAI-compatible chat completions. Модель задаётся через переменные окружения; API key не попадает в отчёт и исходники.

II. Добавлен src/ngsolve_agent/agent_cli.py: запускает D1 или D4 на обычном пользовательском запросе и пишет запрос, архитектуру, provider/model, trace, время/usage LLM-вызовов, solver status и путь к численному отчёту.

III. D4 теперь сохраняет шаги interpret, validate, solve, evaluate в trace.

IV. В README добавлена конфигурация запуска локального Ollama и OpenAI-compatible API.

1.2 Зачем и границы

Раньше D1/D4 принимали только тестовые callback-заглушки. Теперь у проекта есть реальный API-клиентский путь, но FEM остаётся детерминированным и валидируемым: модель может только преобразовать запрос в PoissonTask либо выбрать один из allowlisted D1-инструментов. Произвольный Python-код не генерируется и не запускается. Этот первый LLM-пилот пока ограничен поддерживаемой Poisson-задачей; перенос на Stokes/elasticity/Maxwell - следующий инженерный этап.

1.3 Проверка и ограничение среды

Ruff прошёл для examples, src и tests; compileall завершился без ошибок. Фактический LLM-вызов пока не запускался: локальный Ollama executable/endpoint не обнаружен, NGSOLVE_LLM_API_KE не задан. Значения секретов не читались и не выводились. После настройки доступного endpoint запускать через:

```
uv run python -m ngsolve_agent.agent_cli --architecture d4 --request "..."
uv run python -m ngsolve_agent.agent_cli --architecture d1 --request "..."
```

Сравнительный LLM-run нельзя считать выполненным до подключения модели; в отчёте это будет видно по llm_calls и полю status.

1.4 Состояние локальной установки (проверено повторно)

I. В Windows-среде Codex ollama CLI не найден, а http://127.0.0.1:11434/api/tags не отвечает. Codex не смог установить Ollama: winget завис на загрузке; прямое обращение из терминала к GitHub не прошло из-за недоступности DNS в управляемом окружении. Пользовательское Windows-окружение может иметь другую сетевую политику.

II. Проверка RAM/GPU через CIM и systeminfo также недоступна из управляемого процесса. Поэтому выбран небольшой Qwen3.5 4B для первого прогона без заявления о скорости; официальная карточка Ollama указывает размер около 3.3-4.0 GB.

III. Для запуска из обычного Windows Terminal пользователь должен установить Ollama официальной командой Windows, затем выполнить ollama run qwen3.5:4b и запускающую команду D4 из README. Имя службы OllamaService не проверено и не требуется для штатного запуска из приложения Ollama.

IV. В предоставленном внешнем ответе d4 ошибочно прочитано как степень/уровень метода; в проекте D4 - архитектурная ветка с фиксированным workflow LangGraph.

V. Пока Ollama endpoint отсутствует, выполненных LLM-вызовов и сравнительных результатов нет. Не подменять это заглушками в экспериментальном отчёте.

1.5 Подготовка controlled сравнения

I. Добавлен фиксированный набор data/agent_poisson_cases.json (P1-P5): пять естественно-языковых запросов с эталонными значениями параметров PoissonTask.

II. Добавлен examples/evaluate_agent_architectures.py: запускает выбранный набор на одинаковой модели через D1 и D4, проверяет точное извлечение параметров отдельно от численного passed, считает вызовы/время LLM и пишет comparison JSON.

III. Первый запуск –case P1 предназначен как дешёвая проверка интеграции, прежде чем запускать все десять сочетаний архитектуры и задания.

IV. После добавления comparison runner Ruff прошёл по examples, src, tests, compileall прошёл; –help обеих новых CLI-команд отработал.

V. Проверка окружения: ollama не найден, http://127.0.0.1:11434/api/tags не отвечает, API-key переменная не настроена. Поэтому реальных model calls пока нет.

2 Текущее состояние

I. Проект: Windows, Python 3.12, NGSolve 6.2.2608; зависимости закреплены в pyproject.toml и uv.lock.

II. Запускаются четыре детерминированных FEM benchmark-а: Poisson, нелинейная Neo-Hookean балка, Stokes flow вокруг цилиндра и Maxwell curl-curl eigenproblem на Fichera-геометрии.

III. examples/run_benchmark_suite.py выполняет все четыре задачи и записывает единый отчёт с проверками в results/benchmark_runs/.

IV. D1 и D4 реализованы как каркасы с внедряемыми callback-ами. Реальная LLM-модель ещё не подключена; MCP-интерфейс ещё не добавляли.

V. Дообучение модели не планируется на текущем этапе. Локальная модель - возможный будущий вариант сравнения, а не требование Александра.

3 Запись 2026-11-10 - единый запуск четырёх benchmark-ов

3.1 Что изменено

I. Добавлен examples/run_benchmark_suite.py. Он запускает B0 Poisson, нелинейную упругость, Stokes и Maxwell последовательно, выполняет задачно-зависимые проверки и пишет агрегированный suite.json вместе с per-case JSON и графиками.

II. Добавлена зависимость SciPy в pyproject.toml и uv.lock: NGSolve PINVIT импортирует scipy.linalg при решении Maxwell eigenproblem.

III. В README добавлена команда запуска всего набора.

3.2 Почему

До этого задачи запускались разными командами и имели разные форматы результата. Общий запуск нужен, чтобы позже сравнивать B0, D1 и D4 на тех же задачах и сопоставлять проверки, ошибки и время выполнения.

3.3 Запуск и результат

Команда:

.venv\Scripts\python.exe examples\run_benchmark_suite.py

Итог: все 4 из 4 задач прошли текущие проверки (all_passed: true), NGSolve 6.2.2608, Python 3.12.16. Отчёт этого запуска: results/benchmark_runs/suite-20261110T130937820324Z/suite.json.

Ключевые результаты:

I. Poisson: L2 error 1.6335e-3; свободная невязка 1.1871e-15.

II. Neo-Hookean cantilever: все 50 шагов нагрузки сошлись; свободная невязка 1.1568e-13.

III. Stokes: поток на входе и выходе 0.41; относительный дисбаланс 0; свободная невязка 1.0349e-14. L2-норма дивергенции 0.3157; это измеряемая величина, не нулевой pass/fail порог для Taylor-Hood.

IV. Maxwell/Fichera: 12 положительных собственных значений, 42,103 степени свободы; первая собственная величина 3.2202713671, относительное отличие от значения в tutorial 3.88e-6.

Ruff прошёл для examples, src и tests. Полный suite был реально исполнен; тесты pytest в этой записи повторно не запускались.

3.4 Технические решения и замечания

I. Первоначальный упрощённый 2D Maxwell-вариант не дал ожидаемую первую моду и был отброшен. В журналируемый набор вошёл опубликованный NGSolve 3D Fichera-пример, чтобы сравнивать с его устойчивым справочным спектром.

II. Для установки SciPy использовалась временная папка .uv-cache в рабочей папке; после установки её удалили. OneDrive подтвердил удаление 334 файлов этого кеша. Это были скачиваемые временные данные, не исходники проекта и не результаты.

III. Smart App Control ранее блокировал неподписанные исполняемые файлы окружения; пользователь отключил блокировку самостоятельно. Код проекта обходы системной защиты не добавляет.

4 Сделано ранее

4.1 Исследовательский замысел и ограничения

I. Прочитан NGSolve_Agent_Research_Blueprint.pdf. Основной вопрос проекта - надёжность и воспроизводимость архитектур управления научным расчётом.

II. Зафиксированы сравниваемые архитектуры: B0 - ручной детерминированный решатель; D1 - модель выбирает типизированные инструменты; D4 - фиксированный workflow LangGraph с проверками и ограниченным восстановлением. D3/RAG позже, D2 опционально.

III. По словам Александра Хейнленки нет требования обязательно использовать локальную модель или обучать новую. Начинаем с агента и реальных расчётов; провайдер и local-vs-API сравнение выбираются отдельным этапом.

4.2 Кодовая основа

I. Созданы Poisson B0-решатель, Pydantic-схемы, независимая численная проверка, JSON-отчёт и изображение решения.

II. Созданы D1 и D4 как provider-neutral каркасы с callback-ами; их модельные решения пока проверялись тестовыми заглушками, не подключённой LLM.

III. Исправлены вызовы NGSolve API для вычисления H1-ошибки и свободной невязки.

IV. uv.lock и .venv используются для воспроизводимого Windows-окружения.

5 Следующие шаги

I. Добавить для Stokes ещё один уровень сетки и оценивать изменение divergence/давления, потому что сейчас есть только два уровня.

II. Привести проверки и метрики четырёх задач к формально описанному общему контракту (task ID, параметры, метрики, evaluator version, артефакты), не стирая различия между физическими задачами.

III. Добавить реальные задания NGSolve из учебных материалов: уточнить набор задач, эталонные/проверочные величины и допустимые варианты формулировки.

IV. Подключить один реальный LLM provider к D1 и D4, сохраняя текущий детерминированный FEM solver и evaluator; секреты не хранить в репозитории.

V. Сравнить B0/D1/D4 на одном наборе задач и ограниченном числе повторов. Сохранять model/version, входное условие, выбранные инструменты, trace, повторы, ошибки, численные проверки и время.

VI. Только после первых сравнений решить, нужен ли локальный LLM, RAG (D3) или MCP. MCP подключать после стабилизации обычных Python-функций.

VII. SuiteSparse/Dziekonski и направление Regazzoni ROM вести как отдельные рабочие пакеты, не смешивать с текущим FEM-agent benchmark без отдельной гипотезы.

6 2026-11-10 - первый запуск с локальной Qwen

I. Пользователь установил Ollama и qwen3.5:4b; проверка /api/tags в его PowerShell подтвердила доступность модели. Изолированная среда проекта напрямую не видела локальный порт, поэтому запуски из проекта выполнялись с разрешённым пользователем доступом к 127.0.0.1:11434. Системные настройки Windows не менялись.

II. В Ollama-адаптер добавлены think: false, num_ctx=4096 и num_predict=128, чтобы ограничить непрактично длинный ответ модели на короткие параметры задачи. Ruff и компиляционная проверка Python прошли.

III. Реальный D4-запуск на запрос с maxh=0.1, quadratic elements корректно разобрал poisson_unit_square_smooth, maxh=0.1, order=2; задача прошла. L2 error 1.7011e-4, H1 semi-norm error 1.2633e-2, свободная невязка 2.10e-15, 501 DoF; FEM solver занял 0.37 с. Один вызов модели занял 35.12 с. Отчёт: results/agent_runs/D4-20261110T145923993630Z/agent_run.json.

IV. Выполнено парное сравнение D1/D4 на одном фиксированном кейсе P1 «default settings». D1 закончил расчёт и численную проверку успешно, но модель выбрала maxh=0.1 вместо заданного в fixture значения 0.2; потребовались три вызова модели, 151.08 с суммарно. Поэтому строгая проверка совпадения параметров отметила D1 как fail. D4 выбрал ожидаемые maxh=0.2, order=2, прошёл численную проверку одним вызовом модели за

35.64 с. Сравнение в целом помечено all_passed=false из-за параметрического расхождения D1, а не из-за сбоя FEM. Отчёт: results/agent_benchmarks/agent-compare-20261110T150054102020Z/com

V. Это пока один локальный прогон каждого варианта, не статистический вывод. Следующий шаг - сделать ожидание P1 явным в тексте запроса (убрать неоднозначное «default settings»), затем провести несколько парных повторов; отдельно ограничить число D1-вызовов/время и измерять корректность разбора, число вызовов, задержку, успешность верификации и численные метрики.

6.1 Повтор после уточнения P1

I. Формулировка P1 исправлена: вместо «default settings» текст теперь прямо задаёт maxh=0.2 и quadratic elements. Повторное сравнение D1/D4 прошло полностью: оба выбрали ожидаемые параметры, прошли независимую численную проверку и получили одинаковые метрики (L2 1.6335e-3, свободная невязка 1.19e-15, 125 DoF).

II. D1 использовал 3 вызова модели за 158.99 с; D4 - 1 вызов за 34.85 с. Это только единичная пара на одном кейсе; преимущество по задержке пока предварительное.

III. Отчёт: results/agent_benchmarks/agent-compare-20261110T150531948632Z/comparison.json.

IV. Следующий шаг: запустить тот же явно заданный P1 несколько раз для оценки разброса, после этого расширить сравнение на P2-P5; отдельно проверить лимиты и недопустимые запросы, чтобы убедиться, что агент безопасно отклоняет неподдерживаемую физику.

7 2026-11-10 - P2-P5 и проверка границ поддержки

I. Запущено реальное сравнение D1/D4 на оставшихся кейсах P2-P5 с Ollama qwen3.5:4b. D4 точно воспроизвёл параметры и прошёл численный evaluator во всех четырёх кейсах. Начальная версия D1 прошла FEM и evaluator, но в P4 передала порядок 3 и maxh 0.1, при этом потеряла grid_samples=128 и l2_tolerance=0.03 из запроса; остальные P2, P3, P5 совпали с fixture. Полный исходный отчёт: results/agent_benchmarks/agent-compare-20261110T151057274661Z/comparison.json.

II. Исправлен D1 tool contract: set_task теперь перечисляет все поля задачи, допустимые диапазоны и defaults; системная инструкция требует сохранять все явно запрошенные поля. P4 D1 повторён отдельно и прошёл со всеми четырьмя параметрами. L2 error 3.9001e-6, свободная невязка 2.09e-15, 1096 DoF. Отчёт: results/agent_benchmarks/agent-compare-20261110T153246027822Z/comparison.json.

III. По завершённым прогонам P1-P5, с исправленным контрактом D1, все пары D1/D4 точно воспроизвели параметры и прошли FEM evaluator. Но это один запуск на каждую пару, а не доказательство общей надёжности. D1 по-прежнему делает 3 вызова модели (около 149-212 секунд в этих примерах); D4 делает 1 (около 32-44 секунд).

IV. Выполнен контрольный запрос на 3D Navier-Stokes. До исправления D4 ошибочно объявил задачу passed, подменив её стандартной Poisson-задачей. Для обоих режимов добавлена детерминированная проверка области поддержки до LLM и FEM: запрос должен явно называть Poisson и не должен явно называть распространённые неподдерживаемые PDE (Navier-Stokes, Stokes, Maxwell, elasticity, Helmholtz, heat, wave). Иначе сохраняется unsupported_request; LLM и решатель не вызываются.

V. Добавлены проверки границ для обоих режимов и двух вариантов формулировки запроса. pytest -q –basetemp .pytest-tmp tests/test_agent_cli_scope.py tests/test_workflow.py завершился успешно: 6 passed; Ruff прошёл. Первичная ошибка pytest без –basetemp была вызвана запретом записи во временную папку Windows sandbox; повторный запуск с временным каталогом в проекте прошёл.

VI. Следующий шаг: повторить P1-P5 несколько раз на фиксированной версии модели, затем сравнить точность разбора, число обращений и задержку с разбросом. Лексическая проверка scope намеренно консервативна и пока покрывает только Poisson; при добавлении новых физических постановок её нужно заменить явным registry поддерживаемых task types.

VII. После обновления D1-контракта дополнительно прогнаны scope, D1 и D4 workflow unit tests: 7 passed; Ruff и compileall прошли.

8 2026-11-10 - repeated P4 comparison

I. Добавлен параметр –repeats в examples/evaluate_agent_architectures.py (1-10): каждый прогон сохраняется отдельно; итоговый JSON содержит по каждой паре case/architecture pass rate, exact parameter-match rate, среднее число вызовов, среднее и медианное время. Команда описана в README.

II. P4 прогнан трижды на D1 и трижды на D4 с Ollama qwen3.5:4b. Оба режима: 3/3 численно успешны и 3/3 раз точно восстановили параметры.

III. D1: 3 вызова модели на запуск; среднее время 172.87 с, медиана 141.98 с (значения запусков 241.73, 141.98, 134.92 с). D4: 1 вызов; среднее 30.25 с, медиана 26.81 с (38.13, 26.81, 25.80 с). Наблюдаемый разброс указывает на необходимость большего числа повторов для устойчивой оценки задержки; все шесть текущих результатов успешны.

IV. Отчёт с отдельными прогонами и агрегатами: results/agent_benchmarks/agent-compare-20261110T154149619098Z/comparison.json.

V. Повторы ещё не выполнены на P1-P3 и P5. Следующий технический шаг - решить, нужна ли статистика для всех кейсов или P4 станет расширенным latency/reliability-кейсом, а P1-P5 останутся функциональным smoke benchmark; после решения можно экономно запускать дальнейшие повторы, не умножая без причины медленные D1 вызовы.

9 2026-11-10 - второй тип физической задачи: Stokes

I. Сверка с записью разговора с Александром: его исследовательская идея - найти/собрать agentic framework, который получает описание симуляции, формирует основной driver, запускает научное ПО и постобрабатывает результат; NGSolve/FEM - подходящая стабильная отправная точка. Он прямо предложил сначала изучить литературу и NGSolve, затем делать небольшие эксперименты с AI-агентами и NGSolve. Это exploratory research: он не требовал обучать новую LLM.

II. Реализована первая ступень обобщения: allowlisted registry теперь маршрутизирует две типизированные задачи - manufactured Poisson и фиксированный Stokes tutorial вокруг цилиндра - к разным решателям и отдельным evaluators. Для Stokes проверяются конечность метрик, сходимость, L2 divergence, inlet/outlet mass balance и свободная невязка.

III. examples/stokes_cylinder.py переведён на ту же реализацию src/ngsolve_agent/stokes_solver.py, чтобы tutorial baseline и agent path не расходились. Реальный запуск при maxh=0.05: 3690 DoF, 760 элементов, divergence L2 0.3156747, inlet/outlet flux 0.41, imbalance 0, free residual 1.03e-14.

IV. Реальный D4+Ollama запуск Stokes прошёл одним вызовом модели; модель выбрала ожидаемый Stokes task ID и maxh. Реальный D1+Ollama запуск после уплощения typed tool contract также прошёл, с тремя ожидаемыми решениями (set_task/run_solver/evaluate), без ошибки и восстановления. Первый D1-прогон выявил, что описания полей нельзя кодировать как псевдовложенные поля: модель попыталась передать ключ Stokes fields, схема отвергла его, затем D1 восстановился. Контракт исправлен, чистый повтор это подтвердил.

V. После обобщения выполнен настоящий D4 Poisson regression при maxh=0.1/order=2: прошёл, L2 error 1.7011e-4, residual 2.10e-15.

VI. Полный тестовый набор: 18 passed; Ruff и compileall прошли. README описывает два поддерживаемых agent task types и текущие ограничения.

VII. Граница соответствия запросу Александра: уже есть реальный LLM-driven выбор задачи и параметров, вызов готового NGSolve driver, безопасный запуск фиксированного решателя, независимая оценка и постобработка/отчёт для двух задач. Пока агент НЕ генерирует новый основной driver-код по произвольному описанию; текущие Poisson/Stokes пути - закрытый prototype с allowlist. Следующий исследовательский этап - оценить D3: генерацию NGSolve driver с ограниченным набором шаблонов/инструментов, проверкой до исполнения и независимым evaluator; сравнить, какие запросы требуют новой генерации, а какие выразимы registry. До запуска сгенерированного кода нужен отдельный threat/sandbox design.

10 2026-11-10 - первый D3 driver-generation pilot и сравнение трёх архитектур

I. Добавлен D3. LLM генерирует Python-файл для выбора одного из двух поддерживаемых NGSolve task IDs и передачи параметров. До запуска AST-валидатор разрешает ровно два выражения: импорт run из фиксированного API и один вызов с простыми литералами. Затем исходник пересобирается из Pydantic-валидированных параметров, записывается как canonical driver и запускается отдельным Python -I процессом с минимальным набором переменных окружения. Именно исходный текст модели не исполняется.

II. Первая D3 попытка на P4 была отклонена валидатором, потому что пример в prompt оказался двусмысленным и модель вложила параметры в parameter={...}. Ошибку не обходили: уточнили prompt на плоские именованные аргументы, повторили и получили успешные P4 и S1 запуски. Оба создали candidate и canonical driver, оба прошли независимый evaluator.

III. examples/evaluate_agent_architectures.py теперь поддерживает D1/D3/D4, общую P1-P5/S1 fixture data/agent_task_cases.json, повторяемые прогоны и сквозное время агента. End-to-end включает LLM, построение/проверку driver, subprocess, FEM и evaluator.

IV. Сводное сравнение D1/D3/D4 на P4 и S1 прошло полностью: 6/6 успешных результатов, 6/6 точных совпадений параметров. Сквозное время одного прогона:

a) P4: D1 143.6 s (3 LLM calls), D3 42.0 s (1 call), D4 51.9 s (1 call).

b) S1: D1 128.0 s (3 calls), D3 50.4 s (1 call), D4 59.6 s (1 call).

Один прогон на пару не даёт статистического вывода о превосходстве; D3 измерен только на этих двух кейсах.

V. Сводный отчёт: results/agent_benchmarks/agent-compare-20261110T165408021049Z/comparison.json. D3 candidate и canonical scripts лежат рядом с каждым запуском в подкаталогах d3/.

VI. Полный набор тестов после D3: 24 passed; Ruff, compileall и JSON fixture validation прошли.

10.1 Контрольная сверка с идеей Александра

I. Александр предложил исследовать agentic frameworks и проверить цепочку: описание симуляции -> основной driver -> запуск научного расчёта -> post-processing; начать с NGSolve/FEM и небольших экспериментов, не обязательно создавать или обучать свою LLM.

II. Сейчас проверена архитектурная часть этой цепочки на двух задачах. D1 выбирает typed инструменты, D4 передаёт управление фиксированному workflow, D3 производит

исполняемый main driver, но его содержание пока ограничено вызовом одного заранее реализованного solver API. Поэтому это proof-of-pipeline, а не генерация новой FEM-математики или произвольного решателя.

III. Следующий этап: формализовать исследовательский вопрос и обзор framework/codegen подходов, затем расширять D3 от выбора готового solver к сборке driver из небольшой библиотеки проверенных FEM-компонентов. Сохранять независимые физические checks, ограниченный AST, воспроизводимые запуски и сравнение D1/D3/D4. Сначала - paraphrase/robustness и invalid request tests; потом новые solver components. Не заявлять научную новизну до сравнения с опубликованными agentic scientific-computing системами.

11 2026-11-10 - сверка с гипотезами внешнего ревью по D3

I. Получен текст ревью, в котором автор прямо указал, что исходники и comparison report ему не передали; его security/benchmark замечания были гипотезами, не проверкой кода.

II. По фактической реализации подтверждено: Python -I не является OS sandbox; однако D3 не исполняет исходную строку модели. AST допускает только точный import фиксированного run API, один вызов и простые литеральные keyword arguments; параметры валидируются Pydantic, из них строится новый canonical driver, который и запускается. Подпроцесс получает небольшой env allowlist без NGSOLVE_LLM_API_KEY; произвольные файлы/сеть по OS не ограничены, но исходник модели не имеет выражений, которыми мог бы ими воспользоваться. Уязвимость в самом доверенном solver/dependency этим не закрывается.

III. Подтверждён ресурсный пробел: maxh $>$ 0 допускал патологически мелкую сетку. Заданы нижние пределы maxh $>=$ 0.025 для Poisson и Stokes; ограничения описаны в prompt и схемах.

IV. AST парсер уже имел лимит исходника 16,000 знаков, точную форму из двух statements и отказ от неразрешённых imports/calls/kwargs. Добавлены обработка parser recursion/memory errors и лимит 128 AST nodes. D3 subprocess уже имел timeout 600 секунд.

V. Подтверждён статистический вывод ревью: P4/S1 D1/D3/D4 имеют по одному запуску в общем сравнении; они не дают надёжной оценки частоты отказов. Данных недостаточно для claims о превосходстве архитектуры. D1/D4 ранее повторялись три раза только на P4; D3 - один раз.

VI. Полный тестовый набор после resource/AST изменений: 27 passed; Ruff прошёл. Следующая статистическая работа - сначала несколько повторов D3 на обеих задачах и paraphrase cases; большой 20-run sweep отложен из-за длительных локальных D1 прогонов.

VII. Так как Клод не видел файлы, собран узкий zip для повторного независимого ревью: содержит необходимые модули, тесты, P4/S1 comparison report, candidate/canonical drivers и точное задание ревью. Путь: results/review_bundles/claude_d3_review_20261110.zip.

12 2026-11-10 - проверка ответа Клода и paraphrase robustness

I. Исходный ответ Клода пришёл без приложенных исходников, поэтому он сам обозначил все пункты как гипотезы. По коду проверено: D3 не исполняет raw model source; точная AST-форма и typed параметры обязательны, canonical driver пересобирается; subprocess получает env allowlist без LLM API key. -I не OS sandbox, поэтому код доверенных solver/dependencies всё ещё имеет права текущего пользователя; D3 не даёт произвольному тексту модели доступ к командам/файлам.

II. Закрыт подтверждённый риск слишком малого maxh: схемы ограничивают Poisson и Stokes интервалом от 0.025; нижняя граница синхронизирована с LLM prompts. Добавлены parser recursion/memory handling и лимит 128 AST nodes (source limit 16 KB уже был).

III. Входной scope gate распознал единицу-квадрат только по словам. Новая paraphrase с [0,1]^2 до исправления была отклонена до LLM; добавлено распознавание стандартной записи. После этого D3 на P6 восстановил все параметры и прошёл solve/evaluator. Stokes paraphrase S2 также прошёл. Это зафиксировано в results/agent_benchmarks/agent-compare-20261110T171531076845Z/ и results/agent_benchmarks/agent-compare-20261110T171932139447Z

IV. Полный suite после обновлений: 28 passed; Ruff, compileall и JSON fixture validation passed.

V. Обновлён zip для Клода: results/review_bundles/claude_d3_review_20261110.zip. В нём текущие исходники/тесты, точный review request, основные сравнения, paraphrase reports и generated candidate/canonical drivers. Клоду нужно загрузить этот архив; предыдущий ответ был без файлов.

VI. Статистическая осторожность сохраняется: у D3 пока нет повторов на фиксированном кейсе P4, поэтому скорость/надёжность D3 против D1/D4 не объявлять доказанными.

13 2026-11-11 - независимый запуск Клода и фиксация evaluator policy

I. Получено новое ревью с архивом исходников и независимым запуском solver/evaluator в Linux. Клод сообщил о 30,000 мутациях AST-кандидатов без обхода текущего узкого allowlist; его численные результаты Poisson P4 и Stokes на нескольких сетках полезны как наблюдения, но ревью не запускало полный LLM agent CLI и не проверяло Windows subprocess.

II. Подтверждён критичный дефект: модель могла передать l2_tolerance, divergence_tolerance, flux_imbalance_tolerance и residual_tolerance, тем самым менять критерий собственного успеха. Эти поля удалены из Pydantic task schemas, D1/D3/D4 prompts/contracts и benchmark fixtures. P4/P6 больше не просят настраивать порог точности.

III. Evaluator использует фиксированные пороги: Poisson L2 error <= 2e-2, Stokes relative flux imbalance <= 1e-8, free-DOF residual <= 1e-10. Для Stokes divergence остаётся метрикой, но больше не служит pass/fail gate: для tutorial-задачи нет заданного здесь эталонного решения, а прежний порог был произвольным. Документация отмечает, что эти критерии - pilot checks, а не доказательство точности.

IV. Добавлены проверки контракта: удалённые evaluator thresholds отвергаются схемами; высокий Stokes divergence сам по себе не валит задачу; превышение фиксированного Poisson порога валит её. Тесты и численные baseline после этого изменения ещё требуется прогнать в локальном Windows окружении; текущая запись не утверждает, что этот запуск уже состоялся.

V. Следующий технический этап: прогнать тесты и agent cases после смены evaluator; затем сделать независимую Poisson сеточную/order convergence серию и подобрать Stokes reference/manufactured контроль до расширения benchmark и заявлений о научном качестве.

14 2026-11-11 - полный пакет независимого ревью

I. Выяснена причина замечания Клода о неполном архиве: предыдущий zip был собран как узкий D3 security review bundle и не включал package entry points, lock-файл, все примеры, полный development log и прогоны D1/D4.

II. Собран полный пакет results/review_bundles/claude_review_complete_20261111.zip: весь src/, examples/, tests/, data/, README, DEVELOPMENT_LOG.md, pyproject.toml, uv.lock, а также JSON/driver artifacts P4 3-run D1/D4, paired P4/S1 D1/D3/D4, P6 и S2 paraphrases.

III. В пакет добавлен manifest и обновлённый запрос ревью. В них явно сказано, что historical reports предшествуют новым фиксированным evaluator-порогам и не являются результатом

повторного запуска текущей версии.

IV. Следующие приоритеты из отчёта: (1) запустить текущую версию и перепроверить baseline; (2) добавить регулярный baseline и отрицательные/неоднозначные запросы; (3) прогнать P6/S2 через D1/D4; (4) сделать D3 convergence-study case, где driver действительно задаёт серию расчётов и post-processing наблюдаемого порядка; (5) закрыть subprocess tests/output limits и LLM token cap.